6장 브랜치

소스 코드 이력을 관리하는 버전 외에 깃과 같은 VCS 도구를 사용하는 또 다른 이유는 브랜치 작업 때문입니다. 이 장에서는 브랜치를 자세히 알아봅니다.

6.1 새로운 작업

6.2 실습 준비

6.3 브랜치 생성

6.4 브랜치 확인

6.5 브랜치 이동

6.6 브랜치 공간

6.7 HEAD 포인터

6.8 생성과 이동

6.9 원격 브랜치

6.10 브랜치 전송

6.11 브랜치 삭제

6.12 정리

6.1 새로운 작업

브랜치(branch)는 나뭇가지, 지사, 분점 등 줄기 하나에서 뻗어 나온 갈림길을 의미합니다. 큰 나무 줄기에서 작은 줄기가 뻗어 나오는 것처럼 저장 공간 하나에서 가상의 또 다른 저장 공간을 만드는 것이라고 생각하면 됩니다.

6.1.1 브랜치 작업

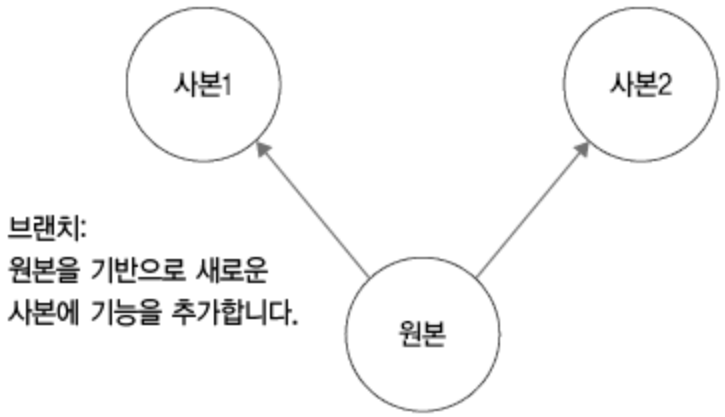
우리는 오랫동안 알게 모르게 브랜치 작업을 해 왔습니다.

보통 새로운 기능을 추가하거나 많은 변경이 예상될 때 작업 폴더를 통째로 복사하고, 복사한 폴더 이름을 변경했습니다. 안정적인 기존 코드는 남겨 두고, 복제된 작업 폴더에서 도전적인 작업들을 하기 위해 코드를 분리합니다.

커밋은 파일의 수정 이력을 관리하는 데 사용한다면, 브랜치는 프로젝트를 독립적으로 관리하는 데 사용합니다. 개발자는 항상 안정된 코드 상태를 유지하고, 개발 중인 작업과 구분하여 관리해야 합니다.

잦은 버그 수정과 새로운 기능을 구현할 때마다 작업 폴더를 복사하는 것은 프로젝트를 유지 관리하는 측면에서 좋지 않습니다. 많은 프로젝트 폴더를 복제하면 향후 코드를 통합하기 어렵습니다.

▼ 그림 6-1 브랜치 분기



깃을 사용하면 프로젝트 작업 폴더를 복사하지 않고 기존 코드와 분리해서 작업할 수 있습니다.

6.1.2 깃 브랜치 특징

깃 브랜치는 기존 폴더를 복제하는 것과 다르게 가상 폴더를 사용하여 개발 작업을 구분합니다. 브랜치는 다음 특징들이 있습니다.

가상 폴더

깃의 브랜치는 작업 폴더를 실제로 복사하지 않고, 가상 폴더로 생성합니다. 외부적으로는 물리적인 파일 하나만 있는 것으로 보입니다.

생성된 작업 폴더는 물리적으로 복제된 구조보다 유연하게 처리할 수 있습니다. 브랜치로 생성된 가상 폴더는 빠르게 공간 이동이 가능합니다. 개발자는 쉽게 가상 폴더인 브랜치를 이동하면서 프로젝트를 수행할 수 있습니다.

독립적인 동작

브랜치를 이용하면 원본 폴더와 분리하여 독립적으로 개발 작업을 수행할 수 있습니다. 기존에는 소스 코드의 작업 폴더를 별도로 생성했습니다. 물리적으로 복사된 각자의 폴더에서 코드를 작업한 후 소스 코드 2개를 다시 하나로 합쳐야 했습니다. 코드를 하나로 합치려면 작업 내역들을 일일이 찾아 정리해야 합니다. 소스 코드를 하나로 통합하는 것은 매우 힘든 작업입니다.

하지만 깃과 같은 버전 관리 시스템을 이용하면 분리된 코드를 좀 더 쉽게 병합할 수 있습니다. 분리된 브랜치에서 소스 코드를 각자 수정한 후 원본 코드에 병합하는 명령만 실행하면 됩니다. 깃의 브랜치는 규모가 큰 코드 수정이나 병합을 처리할 때 매우 유용합니다.

6.2 실습 준비

깃은 기본적으로 main 브랜치를 하나 가지고 있습니다. 그리고 브랜치는 HEAD 포인터를 가지고 있습니다. 실습으로 구체적인 브랜치 작업과 HEAD 동작을 알아보겠습니다.

Note

이 장의 실습 예제는 깃허브 저장소에 공개되어 있습니다. 실습 흐름을 확인할 때 참고합니다.

https://github.com/jinygit/gitstudy06

6.2.1 저장소 생성 및 초기화

브랜치 실습을 위한 환경을 구축합니다.

```
$ cd 메인폴더


$ mkdir gitstudy06 새 폴더 만들기


$ cd gitstudy06
```

```
infoh@DESKTOP MINGW64 /e/gitstudy06


$ git init 저장소 초기화


Initialized empty Git repository in E:/gitstudy06/.git/
```

깃 배시에서 초기화 명령어를 실행합니다. 저장소가 초기화되면 다음과 같이 터미널 프롬프트 창에 현재 브랜치 이름이 같이 출력됩니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)
```

현재 브랜치가 main라는 것을 확인할 수 있습니다. 깃 배시는 리눅스 명령을 사용할 수 있고, 현재 브랜치의 작업 위치도 쉽게 알 수 있습니다.

> 💡 **[참고] GitHub 기본 브랜치 기본값 변경 (master ➔ main)**
> 2020년 10월 이후부터 GitHub을 포함한 대부분의 버전 관리 플랫폼은 새로 생성하는 저장소의 기본 브랜치명을 기존의 `master`에서 `main`으로 기본 변경하였습니다.
> 본 교재에서 자동 가공된 명령어 예시들은 `main` 브랜치로 최신화되어 출력되나, 구버전 Git 클라이언트를 사용하시는 경우 로컬 저장소 생성 시 기본 브랜치가 `master`로 잡힐 수 있습니다.
> 로컬 브랜치명을 main으로 일치시키기 위해 `git branch -M main` 명령어를 활용하시기 바랍니다.

6.2.2 기본 브랜치

모든 커밋과 이력은 브랜치에 기록됩니다. 깃은 최소한 브랜치가 1개 이상 필요합니다. 따라서 저장소를 처음 초기화하면 main 브랜치 하나가 자동으로 생성됩니다. 첫 번째 커밋은 main 브랜치에서 시작합니다. 초기화한 후에 status 명령어를 실행해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git status


On branch main 브랜치 작업 위치


No commits yet


nothing to commit (create/copy files and use "git add" to track)
```

status 명령어의 출력 결과 메시지에서 “On branch main”를 확인할 수 있습니다. 깃에서는 항상 현재 작업하는 브랜치 위치를 확인하는 것이 중요합니다.

또는 branch 명령어로 현재 브랜치를 확인할 수 있습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git branch 브랜치 목록


* main
```

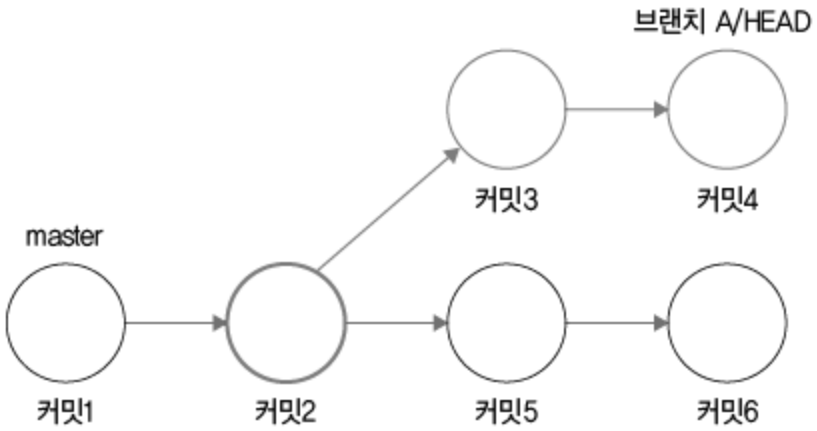
branch 명령어는 생성된 모든 브랜치를 출력합니다. 앞 코드에서는 main라는 이름의 브랜치가 하나 출력되었습니다. 깃에서 기본적으로 선택되는 브랜치는 main입니다. 하지만 꼭 기본값인 main 이름을 그대로 사용할 필요는 없습니다. 통상적으로 깃이 main 브랜치를 자동으로 생성하기 때문에 이를 그대로 많이 사용할 뿐입니다.

6.3 브랜치 생성

브랜치는 가상의 작업 폴더입니다. 처음 깃을 초기화할 때 워킹 디렉터리는 main 브랜치를 생성합니다. 브랜치를 생성하려면 기준이 되는 브랜치 또는 커밋이 하나 있어야 합니다. 그리고 깃은 main 브랜치를 기준으로 새로운 브랜치를 생성합니다.

브랜치는 공통된 커밋을 가리키는 지점입니다. 그리고 브랜치는 커밋처럼 SHA1 해시키를 가리킵니다. 하지만 커밋의 SHA1 해시키는 기억하기가 어렵기 때문에 특정 커밋을 가리키는 별칭을 만드는 것입니다. 이렇게 만든 별칭이 브랜치입니다. 즉, 브랜치를 생성한다는 의미는 기존 브랜치 또는 커밋에 새로운 연결 고리를 하나 더 만드는 것과 같습니다.

▼ 그림 6-2 브랜치는 공통된 커밋을 가리키는 지점



6.3.1 브랜치 생성

처음 생성되는 기본 main 브랜치 외의 브랜치는 사용자가 직접 branch 명령어를 입력하여 생성해야 합니다. 브랜치를 생성한다는 것은 기본적으로 제공되는 main 브랜치 이외에 사용자가 직접 정의한 사용자 브랜치를 이야기하는 것입니다.

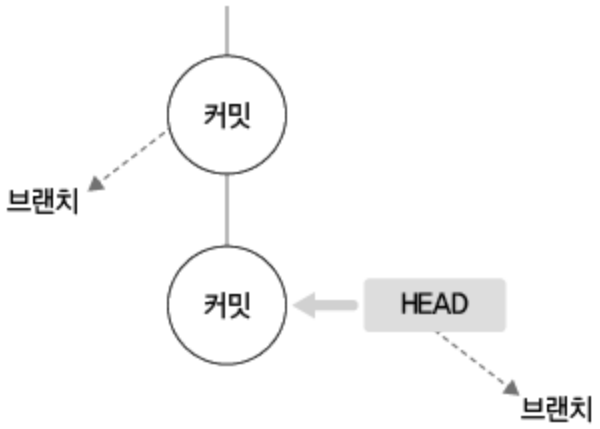
브랜치는 깃에서 또 하나의 개발 분기점을 의미합니다. 새로운 개발 분기점이 필요할 때는 브랜치를 추가로 생성할 수 있습니다. 브랜치 생성 개수에는 제한이 없습니다. 필요한 만큼 여러 브랜치를 생성할 수 있으며, 각 브랜치를 구분하려면 브랜치별로 이름을 지정해야 합니다.

브랜치를 생성할 때는 branch 명령어를 사용합니다.

$ git branch 브랜치이름 커밋ID

branch 명령어 뒤에 브랜치 이름을 인자 값으로 추가합니다. 브랜치 이름만 입력하면 현재 HEAD 포인터를 기준으로 새로운 브랜치를 생성합니다. 직접 커밋 ID 인자 값을 지정하면, 지정한 커밋 ID를 기준으로 브랜치를 생성합니다.

▼ 그림 6-3 브랜치 생성



6.3.2 브랜치 이름

브랜치 이름을 지을 때 사전 예약된 명칭은 따로 없습니다. 다만 해당 브랜치 작업은 알기 쉬운 이름으로 짓는 것이 좋습니다. 또 깃 플로(git flow)에서 정의한 브랜치 이름을 적용하는 것도 좋은 방법입니다.

Note

깃 플로는 유지 보수를 하려고 다수의 브랜치를 생성하여 관리하는 방법입니다. 자세한 내용은 이 책의 학습 사이트 (https://git.jiny.dev)를 참고하세요.

브랜치 이름은 슬래시(/)를 사용하여 계층적인 구조로 만들어서 사용할 수 있습니다. 작성 규칙은 다음과 같습니다.

- 기호(-)로 시작할 수 없습니다.
- 마침표(.)로 시작할 수 없습니다.
- 연속적인 마침표(..)를 포함할 수 없습니다.
- 빈칸, 공백 문자, 물결(~), 캐럿(^), 물음표(?), 별표(*), 대괄호([ ]) 등은 포함할 수 없습니다.
- 아스키 제어 문자는 포함할 수 없습니다.

주의할 점은 브랜치 이름은 중복해서 사용하지 않아야 한다는 것입니다. 이미 생성된 브랜치 이름과 동일한 이름으로 생성한다면 오류가 발생합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git branch footer


fatal: A branch named 'footer' already exists.
```

6.3.3 소스트리 브랜치

이번에는 소스트리로 새로운 브랜치를 하나 생성해 보겠습니다. 소스트리의 새 탭에서 Add 버튼을 클릭합니다. 탐색을 눌러 앞에서 만든 gitstudy06 폴더를 찾아 선택한 후 추가를 누릅니다. 그러면 gitstudy06 저장소와 연결됩니다.

소스트리의 왼쪽에서 브랜치 탭을 확인할 수 있습니다. 브랜치에 모든 브랜치 목록이 같이 출력됩니다.

▼ 그림 6-4 소스트리에서 브랜치 확인



6.4 브랜치 확인

눈치가 빠른 독자라면 앞 실습에서 이미 간단히 브랜치 목록을 확인하는 방법을 다루었음을 눈치챘을 것입니다. 이 절에서는 브랜치 세부 사항을 확인하는 방법을 알아보겠습니다.

6.4.1 간단 브랜치 목록

깃 배시에서 브랜치 목록을 확인하는 방법은 간단합니다. branch 명령어만 입력하면 됩니다. branch 명령어는 단독으로도 실행할 수 있습니다.[2]

$ git branch

현재 브랜치 목록을 확인해 볼까요? git branch 명령어를 실행하면 현재 모든 브랜치가 나열됩니다. 그리고 * 표시가 된 브랜치로 자동 이동합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (feature)


$ git branch 브랜치 목록


* feature 현재의 브랜치(소스트리를 실습할 때 변경된 브랜치 위치)


  footer


  main
```

생성된 전체 브랜치 목록을 출력합니다. 브랜치 이름 앞에는 별표(*)가 붙은 것을 확인할 수 있습니다. 별표(*)는 현재 선택된 브랜치를 의미합니다. 앞에서 우리는 소스트리에서 feature 브랜치를 생성했습니다. 그리고 소스트리 목록에서 ○ 마크가 이동된 것을 확인했습니다. 깃 배시에서는 ○ 마크 대신 별표(*)로 표시됩니다.

6.4.2 브랜치 해시

브랜치는 특정한 커밋의 해시 값(SHA1)을 가리킵니다. 깃의 저수준 명령어인 rev-parse를 사용하면 현재 브랜치가 어떤 커밋 해시 값(SHA1)을 가리키는지 확인할 수 있습니다.

$ git rev-parse 브랜치이름

실습으로 브랜치와 커밋 간 관계를 확인해 보겠습니다. 먼저 커밋 로그를 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (feature)


$ git log 로그 확인


commit d84766c7f87b1d9d234050949c48681ba4e35da8 (HEAD → feature, main, footer)


Author: hojin <infohojin@gmail.com>


Date:   Sat May 11 17:10:02 2019 +0900


    first
```

로그에서 커밋의 d84766c7f87b1d9d234050949c48681ba4e35da8 해시 값(SHA1)을 확인할 수 있습니다.

이번에는 footer 브랜치의 커밋 해시(SHA1)를 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (feature)


$ git rev-parse footer


d84766c7f87b1d9d234050949c48681ba4e35da8
```

브랜치의 해시 값과 브랜치를 생성한 기준 커밋의 해시 값이 동일합니다. 브랜치가 커밋의 해시를 기준으로 생성된다는 것을 다시 한 번 알 수 있습니다.

6.4.3 브랜치 세부 사항 확인

기본적인 branch 명령어는 간단한 브랜치 이름만 출력합니다. 하지만 옵션을 사용하면 좀 더 상세한 브랜치 정보를 얻을 수 있습니다.

branch 명령어 뒤에 -v 또는 -verbose 옵션을 함께 사용하면 브랜치 이름, 커밋 ID, 커밋 메시지를 같이 볼 수 있습니다. 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (feature)


$ git branch -v 브랜치 세부 사항 확인


* feature  d84766c first


  footer  d84766c first


  main  d84766c first
```

이전보다 좀 더 상세한 출력 결과를 얻을 수 있습니다. 이처럼 branch 명령어는 다양한 옵션을 제공합니다. branch 명령어 뒤에 -help 옵션을 추가하면 자세한 사용법을 볼 수 있습니다.

6.5 브랜치 이동

브랜치를 생성했다고 해서 자동으로 생성된 브랜치로 이동하지는 않습니다. 이번에는 생성된 브랜치 간 작업 영역을 이동하는 방법을 알아봅시다.

6.5.1 체크아웃

호텔에서 퇴실할 때 체크아웃이라는 말을 합니다. 체크아웃은 객실을 비우고 떠나는 것을 의미합니다. 즉, 현재 브랜치를 떠나 새로운 브랜치로 들어간다는 의미입니다. 깃에서 브랜치 간 이동할 때는 checkout 명령어를 사용합니다.

$ git checkout 브랜치이름

checkout 명령어로 브랜치 간 이동하면서 실습해 보겠습니다. 주의할 점은 깃은 하나의 워킹 디렉터리만 가지고 있다는 것입니다. 워킹 디렉터리는 선택한 브랜치 하나만 연결되어 있습니다. 즉, 한 브랜치에서만 작업과 커밋을 할 수 있습니다. 따라서 다른 브랜치에서 작업하려면 반드시 브랜치를 변경하여 워킹 디렉터리를 재설정해야 합니다.

Note

워킹 디렉터리에 커밋하지 않은 내용이 있다면 브랜치를 변경할 수 없습니다. 이와 관련된 내용은 7장에서 설명합니다.

6.5.2 브랜치 동작 원리

checkout 명령어로 브랜치가 변경되면 깃은 내부적으로 몇 가지 동작을 수행합니다.

- HEAD 정보는 항상 변경된 브랜치의 마지막 커밋을 가리킵니다. 이처럼 HEAD가 브랜치의 마지막 커밋을 의미하기 때문에 브랜치가 이동하면 HEAD 포인터도 함께 이동합니다.
- 변경된 브랜치로 새로운 작업을 할 수 있도록 워킹 디렉터리를 변경합니다. 브랜치를 변경하려면 기존 브랜치의 워킹 디렉터리를 정리해야 합니다. 기존 브랜치의 워킹 디렉터리를 정리하지 않고서는 브랜치를 변경할 수 없습니다.

6.5.3 소스트리

소스트리에서 브랜치를 변경해 볼까요? 소스트리의 왼쪽에서 변경하고 싶은 브랜치를 선택합니다. 필자는 main 브랜치를 선택하겠습니다. 브랜치를 선택한 상태에서 마우스 오른쪽 버튼을 누릅니다. 다음 화면이 나오면 체크아웃 main... 메뉴를 선택하여 main 브랜치로 변경합니다.

▼ 그림 6-8 main 브랜치로 체크아웃



main 브랜치로 변경되면 main 브랜치 앞에 ○ 마크가 이동합니다.

▼ 그림 6-9 main 브랜치 선택



6.5.4 이전 브랜치

브랜치를 이동하려면 브랜치 이름을 적어 주어야 합니다. 하지만 새로운 브랜치가 아닌 이전 브랜치로 좀 더 편하게 이동할 수 있는 방법이 있습니다. 대시(-)를 사용하는 방법입니다. 리눅스에서 대시(-) 기호는 이전 디렉터리를 의미합니다. 이 대시를 사용하여 쉽게 이전 브랜치로 이동할 수 있습니다.

예

```
$ git checkout -
```

깃 배시에서 git branch -v 명령어를 다시 실행하면 main 브랜치로 변경된 것을 확인할 수 있습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git branch -v 브랜치 목록


  feature  d84766c first


  footer  d84766c first


* main  d84766c first 현재의 브랜치
```

main 브랜치에서 이전 footer 브랜치로 돌아갑시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git checkout - 이전 브랜치로 이동


Switched to branch 'footer'




infoh@DESKTOP MINGW64 /e/gitstudy06 (footer) 이전 브랜치로 되돌아감
```

이전 브랜치인 footer 브랜치로 되돌아간 것을 확인할 수 있습니다.

6.5.5 워킹 디렉터리 정리

체크아웃을 사용하여 브랜치를 이동할 때는 주의 사항이 있습니다. 현재 작업하고 있는 워킹 디렉터리를 정리하고 넘어가야 합니다.

브랜치 동작 원리에서 브랜치가 변경되면 워킹 디렉터리도 같이 변환된다고 했습니다. 따라서 워킹 디렉터리 안에서 작성하던 내용이 있고, 커밋을 하지 않았다면 체크아웃할 때 경고가 발생합니다.

그럼 main 브랜치에서 코드를 수정해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)


$ git checkout main 브랜치 이동


Switched to branch 'main'




infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ code branch.htm VS Code 실행, 코드 수정
```

branch.htm

```
<h1>브랜치 실습을 합니다.</h1>


<h2>main(마스터) 워킹 디렉터리 작업 중</h2>
```

6.6 브랜치 공간

브랜치에서 다른 브랜치로 이동하면 워킹 디렉터리는 변경됩니다. 이 절에서는 브랜치와 워킹 디렉터리가 어떤 관계가 있는지 자세히 알아보겠습니다.

6.6.1 브랜치 로그

현재까지 작업한 로그 기록을 확인해 봅시다. 로그를 출력할 때 브랜치 흐름도 같이 보려면 --graph 옵션을 함께 사용합니다. --graph --all 옵션을 사용하면 모든 로그를 출력합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)


$ git log --graph --all


* commit dcdb1c1fa4ef78bedd8dc13bc267e99391cc9782 (main)


| Author: hojin <infohojin@gmail.com>


| Date:   Sat May 11 18:45:35 2019 +0900


|     main working...


|


* commit d84766c7f87b1d9d234050949c48681ba4e35da8 (HEAD → footer, feature)


  Author: hojin <infohojin@gmail.com>


  Date:   Sat May 11 17:10:02 2019 +0900


      first
```

6.6.2 브랜치 소스 확인

지금까지는 브랜치를 생성한 후 main 브랜치에서만 코드를 수정하고 커밋했습니다. main 브랜치는 dcdb1c1 커밋을 가리킵니다.

footer 브랜치의 소스 코드 내용을 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)

$ cat branch.htm footer의 내용

<h1>브랜치 실습을 합니다.</h1>

이번에는 main 브랜치로 체크아웃하여 소스 코드를 확인해 보겠습니다.

infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)

$ git checkout main 브랜치 이동

Switched to branch 'main'



infoh@DESKTOP MINGW64 /e/gitstudy06 (main)

$ cat branch.htm  main의 내용

<h1>브랜치 실습을 합니다.</h1>

<h2>main(마스터) 워킹 디렉터리 작업 중</h2>
```

두 소스 코드에 차이가 있습니다. 브랜치를 이동하면 변경된 각자 브랜치의 마지막 워킹 디렉터리 상태로 빠르게 변경됩니다. footer는 아직 한 줄짜리 정보를 워킹 디렉터리에 가지고 있는 상태고, main는 두 줄짜리 정보를 워킹 디렉터리에 가지고 있는 상태입니다.

6.7 HEAD 포인터

깃은 객체의 포인터 개념을 사용합니다. 대표적인 객체 포인터는 HEAD입니다. 깃 동작을 정확히 이해하려면 먼저 HEAD가 무엇인지 알아 두는 것이 좋습니다.

HEAD는 4장 커밋에서 잠시 학습했습니다. 이 절에서 좀 더 자세히 알아봅니다.

6.7.1 마지막 커밋

깃은 마지막 커밋 정보가 중요합니다. 깃은 마지막 커밋 정보를 기반으로 새로운 커밋을 생성합니다. 마지막 커밋은 새로운 커밋의 부모 커밋입니다.

시스템이 매번 커밋할 때마다 마지막 커밋 정보를 찾으면 부하가 발생합니다. 깃은 마지막 커밋을 쉽게 확인할 수 있도록 특수한 포인터를 제공합니다. HEAD는 작업 중인 브랜치의 마지막 커밋 ID를 가리키는 참조 포인터입니다.

깃은 마지막 커밋을 가리키는 HEAD 포인터를 부모 커밋으로 대체하여 사용합니다. HEAD 포인터를 사용하여 빠르게 스냅샷을 생성할 수 있습니다.

커밋 로그를 이용하여 HEAD를 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)

$ git checkout footer 브랜치 이동

Switched to branch 'footer'



infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)

$ git log --graph --all 로그 확인

* commit dcdb1c1fa4ef78bedd8dc13bc267e99391cc9782 (main)

| Author: hojin <infohojin@gmail.com>

| Date:   Sat May 11 18:45:35 2019 +0900

|     main working...

|

* commit d84766c7f87b1d9d234050949c48681ba4e35da8 (HEAD → footer, feature) HEAD 위치

  Author: hojin <infohojin@gmail.com>

  Date:   Sat May 11 17:10:02 2019 +0900
```

```
first
```

main 브랜치의 마지막 커밋은 dcdb1c1이고, footer 브랜치의 마지막 커밋은 d84766c입니다. main 브랜치에서 새로운 커밋을 생성할 때 부모 커밋으로 dcdb1c1을 가리키는 HEAD 포인터를 사용합니다. footer 브랜치에서 새로운 커밋을 생성할 때는 d84766c를 가리키는 HEAD 포인터를 사용합니다. 그리고 각 브랜치의 마지막 HEAD 포인터를 사용하여 커밋합니다. 현재 HEAD는 footer 브랜치의 d84766c를 가리킵니다.

6.7.2 브랜치 HEAD

브랜치를 이동하면 HEAD 포인트도 이동됩니다. 브랜치가 여러 개면 HEAD 포인트도 여러 개입니다. 각각의 브랜치마다 마지막 커밋이 다르기 때문입니다. 브랜치마다 마지막 커밋 ID를 가리키는 HEAD 포인터가 하나씩 있습니다.

그럼 브랜치 변경과 HEAD 포인터의 위치 변화를 실습으로 알아보겠습니다. main 브랜치로 이동하여 HEAD 위치를 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (footer)

$ git checkout main 브랜치 이동

Switched to branch 'main'



infoh@DESKTOP MINGW64 /e/gitstudy06 (main)

$ git log --graph --all 로그 출력

* commit dcdb1c1fa4ef78bedd8dc13bc267e99391cc9782 (HEAD → main) HEAD 위치

| Author: hojin <infohojin@gmail.com>

| Date:   Sat May 11 18:45:35 2019 +0900

|     main working...

|

* commit d84766c7f87b1d9d234050949c48681ba4e35da8 (footer, feature)

  Author: hojin <infohojin@gmail.com>

  Date:   Sat May 11 17:10:02 2019 +0900
```

```
first
```

main 브랜치로 변경한 후 HEAD 포인터 위치는 이동된 브랜치의 마지막 커밋 dcdb1c1을 가리킵니다. HEAD 포인터는 브랜치에 따라서 위치가 달라집니다. HEAD는 현재 작업하는 브랜치를 가리키기 때문입니다.

6.7.3 소스트리 HEAD

소스트리에서도 HEAD 포인트 상태를 쉽게 확인할 수 있습니다. 각 브랜치의 마지막 위치를 브랜치 아이콘으로 표시합니다. 예를 들어 각 브랜치마다 마지막 커밋을 HEAD 포인트로 표시합니다.

▼ 그림 6- 11 소스트리에서의 HEAD



HEAD 위치는 각 브랜치의 커밋 개수에 따라 서로 다르게 표시합니다.

6.7.4 상대적 위치

깃의 HEAD 포인터는 내부적으로 커밋을 생성하고 브랜치를 관리하는 데 매우 유용합니다. 또 깃의 다양한 명령어를 입력할 때도 기준점으로 사용합니다. 마지막 커밋 위치인 HEAD를 기준으로 상대적 커밋 위치도 지정할 수 있습니다.

상태적 커밋 위치를 지정할 때는 캐럿(^)과 물결(~) 기호를 같이 사용합니다. ^과 ~은 HEAD를 기준으로 몇 번째인지 상대적인 위치를 지정합니다. 예를 들어 HEAD 포인터 바로 이전의 커밋을 가리킨다면 HEAD^ 또는 HEAD~처럼 사용합니다.

그렇다면 HEAD를 기준으로 이전 3개 위치를 지정하고 싶다면 어떻게 할까요? ^과 ~ 기호는 각각 하나의 상대적 위치를 의미하기 때문에 HEAD^^^, HEAD~~~처럼 사용합니다. 하지만 좀 더 먼 상대적 위치를 지정할 때는 기호가 많아져 이렇게 사용하기 어렵습니다. 이때는 숫자를 사용하여 HEAD^3 또는 HEAD~3처럼 표현합니다.

Note

최근의 특정 위치를 지정할 때는 100644처럼 해시키를 사용하는 것이 편리합니다.

6.7.5 AHEAD, BHEAD

HEAD 앞에 A 또는 B가 붙은 AHEAD와 BHEAD 포인터도 있습니다. 원격 저장소와 연동하여 깃을 관리한다면 브랜치마다 HEAD가 2개 있습니다. 로컬 저장소 브랜치의 HEAD 포인터와 원격 저장소 브랜치의 HEAD 포인터입니다.

원격 저장소와 로컬 저장소는 물리적으로 서로 다른 저장소입니다. 따라서 두 저장소의 마지막 커밋 위치가 일치하지 않을 수 있습니다. 이는 서로 다른 커밋을 가리키는 HEAD 포인터를 가진다는 의미입니다.

AHEAD와 BHEAD는 서로 다른 저장소 간 HEAD 포인터의 위치 차이를 의미합니다. 깃은 항상 원격 저장소의 HEAD와 로컬 저장소의 HEAD를 비교합니다. HEAD는 브랜치마다 다릅니다. 브랜치를 여러 개 운영한다면 다수의 AHEAD와 BHEAD가 생길 수 있습니다.

6.8 생성과 이동

깃의 브랜치를 생성하는 동작과 이동하는 동작은 별개입니다. 브랜치 생성은 branch 명령어를 사용하고, 브랜치 이동은 checkout 명령어를 사용합니다. 브랜치를 생성하면서 동시에 생성된 브랜치로 이동하려면 별도의 명령을 실행해야 합니다.

6.8.1 자동 이동 옵션

브랜치 생성과 이동 명령을 따로 두 번씩 입력하는 것은 불편합니다. 깃은 브랜치 생성과 이동 명령을 한 번에 처리하는 옵션을 제공합니다. 다음과 같이 체크아웃할 때 -b 옵션을 같이 사용하면 브랜치 생성과 이동을 한 번에 할 수 있습니다.

$ git checkout -b 브랜치이름

브랜치 생성과 체크아웃을 동시에 하는 실습을 하겠습니다. -b 옵션으로 브랜치를 생성하면서 동시에 체크아웃도 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git checkout -b hotfix 체크아웃


Switched to a new branch 'hotfix' ① 브랜치 생성




infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix) ② 체크아웃
```

새로운 브랜치가 생성된 메시지와 함께 hotfix 브랜치로 자동 전환되었습니다. 브랜치 목록을 살펴봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)

$ git branch -v 브랜치 목록

  feature d84766c first

  footer  d84766c first

* hotfix  dcdb1c1 main working ... 현재의 브랜치

  main  dcdb1c1 main working ... 브랜치 생성만 했기 때문에 같은 커밋 ID를 가리킴
```

새로운 hotfix 브랜치가 보이고 앞에 별표(*)가 표시된 것을 확인할 수 있습니다.

6.8.2 커밋 이동

브랜치 이동을 좀 더 자세히 알아보겠습니다. 브랜치는 특정한 커밋에 별명을 부여한 것과 같습니다. 일반적으로 브랜치를 생성할 때는 마지막 커밋을 기준으로 합니다. 그리고 커밋 해시 값을 지정한 별칭으로 브랜치 목록에 등록합니다.

이러한 동작 원리로 볼 때 브랜치 이름은 커밋 해시키와 동일합니다. 따라서 브랜치로 이동할 때 꼭 브랜치 이름만 사용할 필요는 없습니다. 브랜치 이름 대신 커밋 해시키를 사용하여 체크아웃할 수도 있습니다.

$ git checkout 커밋해시키

커밋 해시키를 사용하여 체크아웃하려면 해시키를 알고 있어야 합니다. 커밋 해시 값은 40자리로 매우 길어 입력할 때 오류가 많이 생깁니다. 해시키를 전체 사용하지 않고, 앞의 7자리 정도만 사용해도 무리가 없습니다. 이는 유일한 해시 값이 가지는 특징입니다. 7자리만 사용해도 중복될 확률이 적습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)


$ git branch -v 브랜치 목록


  feature d84766c first


  footer  d84766c first


* hotfix  dcdb1c1 main working ...


  main  dcdb1c1 main working ...
```



이 자리만 표시됨

6.8.3 HEAD를 활용한 이동

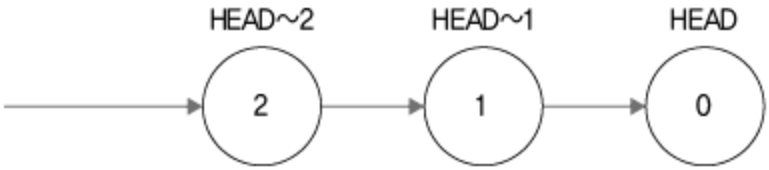
커밋의 해시키를 사용하여 체크아웃하려면 복잡한 해시키를 알고 있어야 합니다. 또 복잡한 영어와 숫자로 표현하므로 입력 오류도 많이 생깁니다. 좀 더 간편하게 HEAD 포인터를 사용하여 체크아웃할 수도 있습니다. 예를 들어 바로 이전 커밋으로 체크아웃하고 싶을 때는 다음 명령을 실행합니다.

예

```
$ git checkout HEAD~1 현재의 한 단계 전
```

마지막 커밋인 HEAD를 기준으로 1단계의 커밋 상태로 이동합니다.

▼ 그림 6-14 HEAD 기준 이동



여러 단계 이전으로 이동하고자 한다면 뒤에 있는 숫자만 바꿉니다.

예

```
$ git checkout HEAD~5
```

6.8.4 돌아오기

커밋 해시키 또는 HEAD를 사용하여 과거의 커밋으로 체크아웃했다면 다시 현재 시점으로 돌아와야 합니다. 간단하게 다시 돌아오는 방법은 대시(-)를 사용하는 것입니다.

예

```
$ git checkout -
```

대시(-)를 사용하면 바로 이전의 브랜치로 복귀합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 ((dcdb1c1 ... ))


$ git checkout - 이전 브랜치로 이동


Switched to branch 'hotfix'




infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)
```



브랜치 복귀

커밋 단계를 여러 번 이동한 후 원래 브랜치(현재)로 되돌아오려면 대시(-) 명령어를 여러 번 실행해야 합니다. 이때는 그냥 직접 브랜치 이름을 입력하는 것이 편리합니다.

예

```
$ git checkout main 특정 브랜치로 복귀
```

이 명령을 실행하면 main 브랜치로 이동합니다. 브랜치 이름을 입력하면 브랜치의 마지막 커밋 위치인 HEAD 포인트로 복귀합니다.

6.9 원격 브랜치

깃은 다수의 개발자와 협업으로 코드를 유지할 수 있습니다. 주요 개발 작업들은 로컬 저장소에서 하지만 협업은 원격 저장소도 공유합니다.

로컬 저장소도 하나의 저장소고, 원격 저장소도 하나의 저장소입니다. 깃은 분산형 버전 관리로서 다수의 저장소를 만들어 연결할 수 있기 때문입니다. 이번에는 브랜치를 이용하여 협업하는 방법을 알아봅니다.

Note

원격 브랜치 실습을 하려면 원격 저장소가 필요합니다. 원격 저장소는 5장에서 설명한 깃허브를 이용합니다.

6.9.1 리모트 브랜치

저장소는 각자의 고유한 브랜치를 생성하고 관리합니다. 원격 저장소에 생성한 브랜치를 리모트 브랜치라고 합니다.

로컬 저장소에 생성한 브랜치는 서버로 공유할 수 있습니다. 원격 저장소와 연결된 로컬 저장소에서 새로운 브랜치를 생성한다고 해서 자동으로 원격 저장소에도 브랜치가 생성되는 것은 아닙니다. 또 원격 저장소에 등록된 브랜치가 자동으로 로컬 저장소를 만들지도 않습니다. 별도 명령을 실행하여 저장소를 동기화해야 합니다.

원격 저장소와 로컬 저장소의 브랜치 이름은 보통 같지만, 반드시 일치하지 않아도 괜찮습니다. 서로 다른 이름으로 브랜치를 연결할 수도 있습니다. 두 저장소는 서로 다른 브랜치로 운영·관리할 수 있습니다.

리모트 브랜치는 보통 별칭/브랜치 이름 형태입니다.

6.9.2 실습 준비

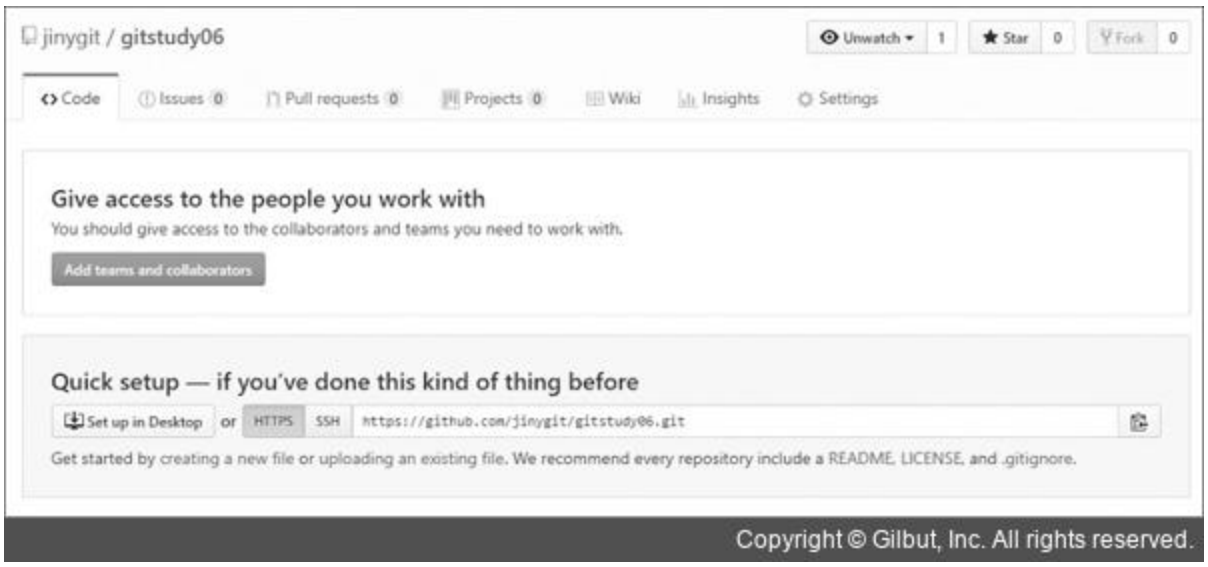
실습을 위해 깃허브에 새로운 실습 저장소를 하나 생성합시다. 저장소는 5장에서 설명했듯이, 반드시 자신의 계정으로 로그인합니다. New 버튼 또는 + > New repository를 클릭합니다. Repository name에 깃허브의 저장소 이름을 입력합니다. 필자는 이름을 gitstudy06으로 했습니다.

▼ 그림 6-15 깃허브에서 새 저장소 생성



모두 입력한 후 Create repository를 누르면 다음 화면이 나오고 저장소 주소를 확인할 수 있습니다.

▼ 그림 6-16 새 저장소의 주소



6.9.3 브랜치 추적

깃의 브랜치는 특정 커밋 해시 값을 가리키는 포인터입니다. 리모트 브랜치 또한 원격 저장소의 브랜치를 가리키는 포인터입니다. 원격 저장소의 브랜치를 가리키는 것을 브랜치 추적이라고 합니다.

다른 용어로 추적 브랜치를 트래킹 브랜치라고 합니다. 트래킹 브랜치는 원격 브랜치를 가리키는 북마크와 같습니다. 추적 브랜치는 원격 브랜치의 마지막 커밋 해시 값을 가리킵니다. 이 포인터 정보는 .git/refs 폴더 안에 저장되어 있습니다.

예

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)


$ ls .git/refs/ 원격 브랜치 정보


heads  tags
```

로컬 저장소가 원격 저장소와 연결될 때 원격 브랜치의 트래킹 정보는 자동으로 갱신됩니다. 로컬 저장소는 마지막으로 연결된 리모트 브랜치의 커밋 해시 값을 항상 가지고 있습니다.

6.9.4 브랜치 업로드

지금까지 로컬 저장소에서만 브랜치를 생성했습니다. 로컬 저장소의 브랜치 정보는 원격 저장소에 자동으로 등록되지 않습니다.

등록된 원격 저장소의 리모트 브랜치는 remote show 명령어로 확인할 수 있습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)


$ git remote show origin 원격 브랜치


* remote origin


  Fetch URL: https://github.com/jinygit/gitstudy06.git


  Push  URL: https://github.com/jinygit/gitstudy06.git


  HEAD branch: (unknown)
```

현재는 원격 저장소를 등록만 했기 때문에 아직 리모트 브랜치는 없습니다. 리모트 브랜치는 서버 간에 통신을 하고 나서 생성됩니다.

로컬 저장소의 브랜치를 원격 저장소에 동기화하려면 푸시 작업을 해야 합니다.

$ git push 원격저장소별칭 브랜치이름

6.9.5 이름이 다른 브랜치

일반적으로 로컬 저장소의 브랜치 이름과 원격 저장소의 브랜치 이름은 동일하게 사용합니다. 하지만 반드시 이름이 동일할 필요는 없습니다. 가끔은 동일한 브랜치 이름을 사용하기 어려울 때가 있습니다. 예를 들어 다른 개발자가 만든 원격 서버의 브랜치를 테스트하려고 할 때 자신의 브랜치 이름과 동일하면 충돌이 생깁니다. 이때는 어떻게 해야 할까요?

깃은 서로 다른 로컬 브랜치와 리모트 브랜치를 수동으로 지정하여 연결할 수 있습니다. 브랜치를 직접 수동으로 지정할 때는 콜론(:)으로 구분합니다.

$ git push origin 브랜치이름:새로운브랜치

현재 브랜치를 서버(origin)의 새로운 브랜치 이름으로 전송하라는 의미입니다. 이번에는 로컬 저장소의 feature 브랜치를 원격 저장소의 function 브랜치로 푸시해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)

$ git push -u origin feature:function 다른 이름으로 브랜치 전송

Total 0 (delta 0), reused 0 (delta 0)

remote:

remote: Create a pull request for 'function' on GitHub by visiting:

remote:      https://github.com/jinygit/gitstudy06/pull/new/function

remote:

To https://github.com/jinygit/gitstudy06.git

 * [new branch]      feature → function 리모트 브랜치 생성

Branch 'feature' set up to track remote branch 'function' from 'origin'.
```

6.9.6 업스트림 트래킹

업스트림(upstream)은 브랜치 추적을 다르게 표현한 것입니다. 앞에서 원격 저장소에 새로운 리모트 브랜치를 생성하는 방법을 알아보았습니다.

리모트 브랜치는 브랜치 이름을 동일하게 생성할 수도 있고, 다른 이름으로 생성할 수도 있습니다. 이처럼 로컬 저장소의 브랜치와 원격 저장소의 브랜치는 업로드할 수 있도록 매칭되어 있습니다. 이러한 매칭을 업스트림 트래킹이라고 합니다.

▼ 그림 6-21 업스트림 트래킹



트래킹 브랜치(업스트림)는 리모트 브랜치와 로컬 브랜치를 연결해 주는 중간 다리 역할을 합니다. clone 명령어로 저장소를 복제할 때 원격 저장소에 등록된 트래킹 브랜치들을 자동으로 함께 설정합니다.

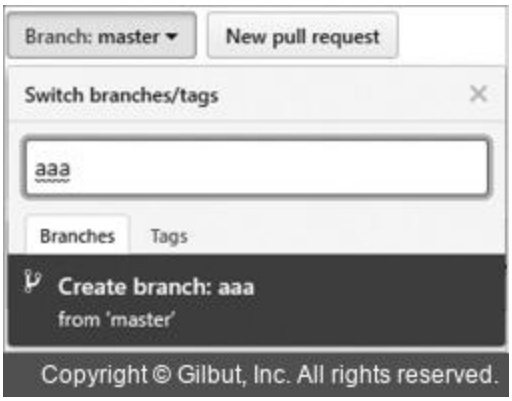
6.9.7 원격 브랜치 복사

원격 저장소와 로컬 저장소의 브랜치 목록은 서로 다를 수 있습니다. 다른 개발자가 원격 저장소에 새로운 리모트 브랜치를 생성할 수 있기 때문입니다. 이렇게 생성된 원격 저장소의 리모트 브랜치를 이용해서 로컬 저장소에도 새로운 브랜치를 생성하여 동기화할 수 있습니다.

$ git checkout -b 새이름 origin/브랜치이름

실습을 위해 깃허브 원격 저장소에서 새로운 브랜치를 하나 생성하겠습니다. 깃허브 저장소에서 브랜치 항목(Branch: main)을 선택합니다. 입력란에 원하는 새 이름을 입력합니다. 책에서는 aaa로 만들겠습니다. 입력란에 aaa를 입력한 후 Create branch: aaa를 클릭합니다.

▼ 그림 6-25 aaa 브랜치 생성



6.9.8 업스트림 연결

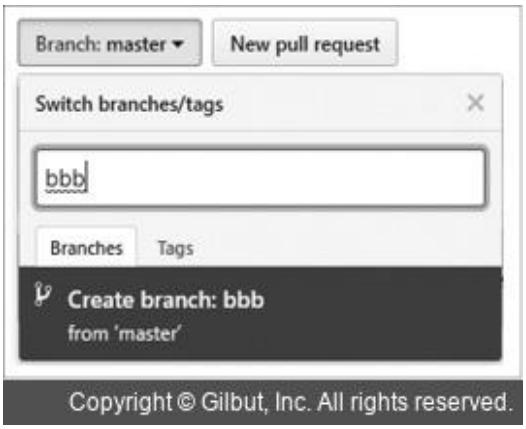
기존에 있는 브랜치를 업스트림으로 직접 설정할 수도 있습니다. 브랜치를 생성한 후 직접 트래킹 브랜치를 지정했습니다. 업스트림을 직접 설정하면 원격 저장소로 트래킹 브랜치가 설정됩니다.

$ git branch -u origin/브랜치이름

-u 옵션은 --set-upstream-to의 약자입니다. 기존 브랜치를 특정 원격 브랜치로 추적합니다. 한 번만 실행하면 이후로는 계속 업스트림으로 설정되어 작업할 수 있습니다.

실습을 위해 깃허브에 새로운 브랜치를 추가해 봅시다. 깃허브 페이지에서 Branch: main를 클릭하고 bbb를 입력합니다. Create branch: bbb를 클릭하여 bbb 브랜치를 생성합니다.

▼ 그림 6-30 bbb 브랜치 생성



깃허브에서 직접 브랜치를 생성

6.10 브랜치 전송

로컬 저장소의 브랜치를 원격 저장소로 전송하는 다른 방법을 살펴봅시다.

6.10.1 브랜치 푸시

깃의 푸시 작업은 로컬 저장소의 파일들을 원격 저장소로 전송합니다. 파일뿐만 아니라 브랜치 정보와 커밋까지 모두 전송합니다.

처음으로 로컬 저장소에 새로운 원격 저장소가 등록되면 다음과 같이 push 명령어를 사용할 때 오류 메시지가 출력됩니다.

예

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git push 원격 서버 전송


fatal: The current branch main has no upstream branch.


To push the current branch and set the remote as upstream, use




    git push --set-upstream origin main 오류 메시지
```

처음에는 커밋과 브랜치를 푸시하는 데 업스트림 설정이 필요합니다. 원격 저장소 연결만으로 업스트림이 자동으로 설정되지는 않습니다. 이는 깃이 원격 저장소의 어느 브랜치에 어떻게 푸시해야 할지 모르기 때문입니다.

그래서 처음 푸시 작업을 하면 깃은 사용자에게 업스트림이 설정되지 않았다는 오류 메시지를 출력합니다. 그리고 친절하게 설정하는 방법도 안내합니다. 오류 메시지에서 표시된 명령어를 입력하면 됩니다.

처음에는 다음과 같이 수동으로 트래킹 브랜치와 업스트림 설정을 해야 합니다.

예

```
$ git push --set-upstream origin main
```

6.10.2 브랜치 페치

리모트 브랜치 페치는 일반적인 커밋 페치와 동일합니다. 리모트 브랜치를 페치한다고 해서 자동으로 로컬 저장소에 새로운 브랜치가 생성되지는 않습니다. 페치 동작은 원격 저장소에서 리모트 브랜치 내용을 내려받기만 할 뿐이지 자동으로 병합하지 않기 때문입니다.

리모트 브랜치가 페치되면 깃은 단순히 원격저장소별칭/브랜치 포인터만 생성합니다. 원격 저장소에서 페치된 커밋들을 새로운 로컬 브랜치로 반영하려면 병합 명령을 실행해야 합니다.

$ git merge 원격저장소별칭/브랜치이름

가끔은 페치된 브랜치를 병합하지 않고 테스트만 하고 싶을 때도 있습니다. 이때는 원격 브랜치의 포인터를 사용하여 임시 브랜치를 생성하거나 직접 체크아웃할 수 있습니다.

$ git checkout -b 임시브랜치이름 origin/브랜치이름

6.11 브랜치 삭제

생성된 브랜치를 삭제하는 것은 생각보다 간단합니다. 하지만 브랜치를 삭제하는 것은 해당 브랜치 내용과 커밋을 모두 삭제하는 것입니다. 따라서 삭제 명령을 실행할 때는 주의해야 합니다. 브랜치 삭제는 크게 스테이지 상태에 따라 달라집니다.

가장 먼저 주의할 점은 현재 자신이 있는 브랜치는 삭제할 수 없다는 것입니다. 브랜치를 삭제하려면 main 브랜치나 다른 브랜치로 잠시 이동해야 합니다. 예를 들어 앞에서 만든 로컬 저장소의 bug 브랜치를 삭제한다고 합시다.

예

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (bug)


$ git branch -d bug 자신의 브랜치 삭제


error: Cannot delete branch 'aaa' checked out at 'E:/gitstudy06' 오류 발생
```

현재 자신의 브랜치에서 삭제 명령어를 실행하면 오류 메시지가 출력됩니다. 어느 브랜치로 체크아웃될지 모르기 때문입니다. 그래서 삭제하고자 할 때는 다른 브랜치로 이동해서 삭제해야 합니다.

6.11.1 일반적인 삭제 방법

일반적으로 브랜치를 삭제할 때는 -d 옵션을 사용합니다.

$ git branch -d 브랜치이름

-d 옵션은 스테이지 상태가 깨끗할 때만 삭제를 허용합니다. 워킹 디렉터리에 작업한 기록이 있거나 add 명령어로 스테이지의 인덱스가 변경된 상태라면 삭제하지 않습니다. 삭제하려면 반드시 최종 상태가 커밋되어 깨끗한 스테이지 상태여야 합니다. 또 병합되지 않은 브랜치는 -d 옵션으로 삭제할 수 없습니다.

-d 옵션을 사용하여 footer 브랜치를 삭제해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (bug)


$ git branch -d footer 브랜치 삭제


Deleted branch footer (was cc66812).
```

잘 삭제되었습니다.

6.11.2 강제로 삭제하는 방법

워킹 디렉터리 또는 스테이지에 추가 커밋 작업이 남아 있다면 일반적인 방법으로는 브랜치를 삭제할 수 없습니다. 이때는 강제로 삭제해야 합니다.

실습을 위해 hotfix 브랜치로 이동한 후 branch.htm 파일을 수정하고 저장합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (bug)


$ git checkout hotfix 브랜치 이동


Switched to branch 'hotfix'




infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)


$ code branch.htm VS Code 실행
```

branch.htm

```
<h1>브랜치 실습을 합니다.</h1>


<h2>main(마스터) 워킹 디렉터리 작업 중</h2>


<h3>hotfix 수정 작업입니다.</h3>
```

수정했다면 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (hotfix)

$ git commit -am "hotfix working" 등록 및 커밋

[hotfix 0225e85] hotfix working

 1 file changed, 2 insertions(+), 1 deletion(-)
```

6.11.3 소스트리에서 삭제하는 방법

소스트리에서도 브랜치를 삭제할 수 있습니다. 먼저 소스트리의 왼쪽에서 브랜치 목록 중 삭제하고 싶은 것을 하나 선택합니다. 브랜치를 선택한 상태에서 마우스 오른쪽 버튼을 누릅니다. 예를 들어 다음과 같이 브랜치를 삭제할 수 있는 메뉴가 나오면 feature 삭제를 선택합니다.

▼ 그림 6-32 소스트리에서 브랜치 삭제



6.11.4 리모트 브랜치를 삭제하는 방법

지금까지는 로컬에 생성된 브랜치를 삭제하는 방법을 알아보았습니다. 그렇다면 원격 저장소의 브랜치는 어떻게 삭제해야 할까요? 원격 브랜치를 삭제하려면 먼저 삭제 명령을 푸시해야 합니다.

$ git push origin --delete 리모트브랜치이름

그럼 깃허브의 aaa 브랜치를 삭제해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy06 (main)


$ git push origin --delete aaa 삭제 명령 전송


To https://github.com/jinygit/gitstudy06.git


 - [deleted]         aaa
```

깃허브 페이지에서 aaa 브랜치가 삭제된 것을 확인할 수 있습니다. 원격 저장소의 브랜치를 삭제하면 리모트 브랜치에 기록된 커밋도 모두 삭제됩니다. 따라서 함께 사용하는 브랜치라면 신중하게 삭제하는 것이 좋습니다.

6.12 정리

이 장에서는 브랜치 개념과 여러 브랜치를 이동하고 작업하는 방법들을 알아보았습니다. 브랜치는 기존 코드를 가상으로 분리합니다. 분리해서 격리된 브랜치는 상호 간섭 없이 별개 작업을 수행할 수 있습니다. 새로운 프로젝트로 발전시켜 나아갈 때 브랜치 기능은 매우 유용합니다.

깃의 브랜치는 다른 개발자와 협업하여 프로젝트를 진행할 때도 매우 유용합니다. 서로 간섭 없이 코드를 개선하고, 나아가 병합도 가능하기 때문입니다. 처음에는 브랜치를 좀 어렵게 느낄 수 있습니다. 하지만 깃의 원리와 개념을 잘 이해하고, 자주 사용하다 보면 생각보다 그리 어렵지 않습니다.

1 작업 사본 부모는 HEAD를 기준으로 생성하는 것이고, 명시적 커밋은 특정 커밋으로 브랜치를 생성하는 것입니다.

2 branch 명령어를 단독으로 입력하면 목록만 출력합니다. 다른 추가 동작은 하지 않습니다.